3D Tic-Tac-Toe Implementation in JAVA and Opening Strategies

Xiaowen Song

April 2025

Abstract

This project investigates the strategic importance of opening moves in 3D Tic-Tac-Toe using automated players implemented in Java. The game extends the classic Tic-Tac-Toe into three dimensions, forming a 3×3×3 cube with 27 possible positions and 49 winning lines. We developed a fully functional Java-based simulation framework supporting both random play and intelligent strategies: a Minimax AI with alpha-beta pruning and a Greedy heuristic AI that evaluates board states based on immediate gains. We conducted a comprehensive set of experiments, simulating games from every possible first-move position under different scenarios. Each game was simulated 1,000 times per position, and the win rates of the Computer AI were recorded and visualized in heat maps. Results show that certain geometrically central positions, especially the exact center (1,1,1), consistently provide a stronger starting advantage. Furthermore, the effectiveness of each opening varied substantially depending on the strategy matchup.

1 Introduction

3D Tic-Tac-Toe transforms a familiar 2D game into a richer, more challenging environment, with 27 positions and 49 potential winning lines. While often dismissed as a solved game in 2D, its 3D version introduces non-trivial decision-making and spatial strategy, making it a useful model for studying AI behavior.

This project explores how the choice of the first move, or opening, affects gameplay when different strategies are used. Specifically, we compare a Minimax agent, which simulates future outcomes, with a Greedy agent, which evaluates moves based on immediate scoring. By simulating games across all 27 opening positions and four strategic matchups, we aim to determine which positions are strongest and how strategy influences their effectiveness.

Understanding the interaction between geometry, strategy depth, and move order not only improves AI performance in games but also provides insight into broader decision-making systems.

2 Literature Review

2.1 History Background and Theoretical Fundatinos

The variations of tic-tac-toe games have been an interesting topic in the study of positional games. Oren Patashnik's 1980 paper [1] provides innovative strategies for designing automotive gameplayers in a 4x4x4 tic-tac-toe which he referred to as qubic. Patashnik resolved a longstanding conjecture by proving that the first player can force a win under the optimal play principle. This conclusion challenged previous assumptions derived from the theorems of Hales and Jewett (1963) and Erd and Selfridge (1973), which believed that certain positional games with specific point-to-line ratios or dimensional constraints could result in draws.

Patashnik's method leveraged automorphisms (symmetrical transformations and preserving the cube's structure). He reduced the game tree from over 249,984 3-position possible sequences to only 7 distinct nodes. This reduction was critical given the computational limitations of the era; the proof required 1,500 hours of processing on a PDP-10 mainframe. Human intuition guided strategic decisions at critical moments, while automated algorithms handled tactical sequences. The hybrid human-computer approach foreshadowed modern AI techniques, where human expertise complements algorithmic efficiency.

Patashnik's work had broader implications beyond the Qubic. By demonstrating that "class 2" games (where draws exist but first-player wins prevail) are feasible, he expanded the classification of k^n-games and challenged assumptions about equilibrium in positional games. Independent verification by Ken Thompson, who replicated the result using a distinct algorithm, bolstered the credibility of computational methods in mathematics.

This historical context underscores the complexity of 3D tic-tac-toe and highlights the necessity of innovative strategies for automating gameplay. Patashnik's work remains foundational for understanding the interplay between combinatorial complexity and computational feasibility in adversarial games.

2.2 Minimax Algorithm

The Minimax algorithm, a cornerstone of adversarial game AI, has been extensively applied to tic-tac-toe variants. Its core principle is maximizing the minimum gain while minimizing the maximum loss, which aligns perfectly with deterministic games where players alternate turns. Bradley Bogenschutz's implementation of 3D tic-tac-toe in C++ [2] exemplifies this approach. By pairing minimax with alpha-beta pruning, Bogenschutz reduced the search space from 16 million to 4,096 nodes for a 4×4×4 board, enabling real-time decision-making on modest hardware.

The Key optimizations in Bogenschutz's paper include heuristic evaluation, depth limitation, and recursive memory management. Heuristic evaluation involves a scoring system that prioritizes lines with higher concentrations of the AI's markers, enabling rapid threat assessment. Depth limitation is by restricting the search to 4-ply maintained strategic depth while avoiding combinatorial explosion. Finally, manage recursive memory by avoiding full tree storage to mitigate memory constraints.

Rajab and Hassan's work uses static evaluators to assess board states [3]. They further

leverage symmetry reductions (rotations, reflections) to collapse equivalent board states, a strategy employed in combinatorial game theory to streamline state-space exploration.

The Indonesian study by Thamrin and Pamungkas further discussed the minimax's limitations in larger grids [4]. For 4×4 2D tic-tac-toe, pure minimax exhibited exponential time complexity ($O(x^n)$), which required over 3 hours for 12 empty cells. To address this, the authors proposed a hybrid model: random moves for early-game simplicity and minimax for later-game precision. This approach balanced computational loading and player engagement, achieving a 10% human win rate while preserving the challenge.

However, these adaptations reveal some trade-offs. It is clear that pure minimax remains impractical for 4×4×4 boards without aggressive pruning or depth limits.

2.3 Neural Network

Neural networks (NNs) have emerged as powerful tools for automating tic-tac-toe, particularly in learning value functions and board evaluations. Steeg, Drugan, and Wiering's work on "Temporal Difference Learning for 3D Tic-Tac-Toe" [5] exemplifies this paradigm. Their structured multilayer perceptron (MLP), designed with sparse connections aligned to 76 winning lines, achieved superior performance against fixed opponents. The network's architecture, 64 inputs (board states), 304 hidden units (4 per line), and 1 output (state value), leveraged domain-specific knowledge to accelerate learning.

Comparatively, Dalffa, Nasser, and Naser's simpler ANN (9 inputs, 3 hidden neurons) achieved 99.38% accuracy on 3×3 tic-tac-toe using supervised learning [6]. However, this approach struggled with overfitting due to limited data (958 samples) and lacked scalability to 3D variants. Sridhar and Shetty's deep neural network (DNN) with five hidden layers achieved 98.9% accuracy on the same dataset [7], suggesting that depth enhances pattern extraction even in simple games.

The critical insights from neural network implementations include embedding game rules into network design to improve convergence. Neural network requires extensive data for generalization and struggles with transparency compared to rule-based systems.

While neural networks excel in pattern recognition, their computational demands and "black-box" nature pose challenges for real-time 3D tic-tac-toe gameplay. Hybrid models, which combine neural networks with symbolic reasoning (e.g., minimax for critical moves), might mitigate these issues.

2.4 Winning Strategy and Fairness of the Game

A central challenge in automating tic-tac-toe lies in balancing optimal play principles with an engaging gameplay experience. Patashnik's proof of a first-player win in Qubic underscores the inherent unfairness in deterministic 3D variants: perfect move by the AI guarantees victory, rendering human opponents obsolete. The percentages for the first player are close to 99% in some cases [8]. To address this, Thamrin and Pamungkas introduced a hybrid model for 4×4 tic-tac-toe, where the AI alternates between random and minimax-driven moves. This approach yielded a 10% human win rate, fostering challenge without sacrificing computational feasibility.

Key fairness mechanisms may include introducing stochasticity to prevent predictability, as seen in the Indonesian study, and adjusting search depth or pruning intensity based on the player's difficulty selected.

3 Implementation of the 3D Tic-Tac-Toe Game in Java

3.1 Representation

The implementation of the 3D Tic-Tac-Toe game follows a standard 3x3x3 board configuration, which introduces a significant increase in complexity and challenge compared to the traditional 2D Tic-Tac-Toe game. The game is implemented in Java, with the major game logic contained in the class TTT3D.java. The board is represented as a three-dimensional array of integers or characters to keep track of the state of each cell. Each cell can be in one of three states: empty '-', occupied by player X, or occupied by player O.

To facilitate GUI interaction and visualization of the game in three dimensions, the board is linked to a graphical user interface, which updates after each move. The program supports both real human manual operation in GUI and automatic simulation for games between AI agents for experimental purposes. In the GUI, a real human player can choose whether to go first, the difficulty of the game (easy/medium/hard), and which symbol to use. The program architecture separates the visual components from the main game logic, ensuring modularity and extensibility. To achieve automatic simulation, the game logic can run independently of the GUI using simulation methods added later in the TTT3D.java file. The automated simulation is implemented in SimulationRunner.java.

Internally, each move is validated based on whether the cell is empty, and the win conditions are checked after each move to ensure no extra move is made. Given the 3D nature of the board, the win-checking logic considers all possible straight-line paths:

- arows within each 2D layer: 3 rows $\times$ 3 layers = 9 lines
- columns within each 2D layer: 3 columns $\times$ 3 layers = 9 lines
- pillars vertical through 2D layers: 3 per row $\times$ 3 per column = 9 lines
- diagonals within each 2D layer: 2 diagonals $\times$ 3 layers = 6 lines
- diagonals that are vertical across layers: 2 per row $\times$ 3 rows + 2 per column $\times$ 3 columns = 12 lines
- diagonals that span corner to the opposite corner through the center: 4 lines

With a total of 49 possible winning lines, 3x3x3 3D Tic-Tac-Toe adds significant branching and strategic depth to the game.

3.2 Algorithms

The AI players in the implementation are designed to make moves based on two different algorithms: random move selection for simpler models and Minimax algorithm enhanced

by alpha-beta pruning under a difficult model. The random move strategy serves as a baseline for comparison, while the Minimax algorithm aims to evaluate and optimize moves to maximize wins or minimize losses.

A Random Move Strategy

The simplest AI player will choose moves randomly. This implementation is used when the real human player chooses to play in the easier models and lets the AI player go first. If the AI agent uses the Minimax algorithm and moves first simultaneously, it will be almost impossible for a real human player to win. This implementation is also used for benchmarking purposes and serves as a control group in simulation experiments. The random strategy uses a uniform distribution to select the remaining legal moves. Although not optimal from a strategic perspective, the random move generation method can provide insights into the relative advantages of openings on a level playing field.

B Heuristic Mode

The evaluation function estimates the quality of non-terminal states. In the easy difficulty, the AI agent bypasses the recursive search logic and instead relies on a simple greedy evaluation using the heuristic(player) function. This heuristic calculates the number of potential winning lines remaining for player X minus those available to its opponent O. Estimates the quality of each move based solely on the current board state, without projecting future moves. Although faster, this strategy lacks strategic foresight and often fails against players who can plan more than one move ahead.

In medium and hard difficulty, the AI agent follows full recursive simulation with terminal evaluation using the function lookAhead(player, alpha, beta). This heuristic explores the tree until a terminal node or maximum depth is reached. The depth of the search is set to two in the medium mode and six in the hard mode to provide more challenge.

C Minimax Implementation

The Minimax algorithm is implemented as a recursive function that evaluates all possible future game states resulting from a move. It assumes that the computer tries to maximize its winning potential while minimizing the opponent's chances of winning. In each recursive call, the algorithm evaluates whether the current board state is terminal (win or loss). If the game is not over, the algorithm recursively explores all valid moves, switching roles between the maximizer and minimizer at each level.

Because of the exponential growth in the number of possible board states in a 3D game, depth-limited recursion is used there. The computerPlays() method is the entry point for the Minimax implementation. It iterates over all possible moves, checks for an immediate win, and calls lookAhead() to evaluate the moves further by Alpha-beta pruning.

D Alpha-Beta Pruning

Alpha-beta pruning is used to optimize the Minimax algorithm by eliminating branches of the game tree that do not need to be explored. If the algorithm discovers that a move

leads to a worse outcome than a previously examined move, it abandons further evaluation of that move. This optimization significantly reduces the computational time and allows deeper exploration of the game tree within a reasonable time limit.

The alpha (best already explored option for the maximizer) and beta (best already explored option for the minimizer) values are passed down the recursion and updated based on the outcomes of child nodes. If a branch is found that leads to a worse score than a previously analyzed move, further branches are pruned, improving performance without sacrificing decision quality.

4 Experiments

4.1 Design

To analyze the strength of the opening move in 3D Tic-Tac-Toe, we designed a set of experiments that automatically simulate games between two computer-controlled AIs under various strategic conditions. This allowed us to evaluate which initial moves confer the greatest advantage to the player who starts at that position. The key focus of our experiments is to assess the win rate of the computer AI when forced to make its first move on a specific cell in the 3×3×3 board.

The board is composed of 27 cells, indexed by their (layer, row, column) coordinates. For each cell, the computer AI is forced to play first on that position, and then the remainder of the game is played automatically between the two AI players using different strategy combinations. In a single simulation, each cell performs 1000 games. One thing to mention is that although the simulation uses the term "human play", it is essentially an AI player. To differ from the first player, the second player is referred to as **human AI** which always goes second. By repeating games for each cell across different configurations, we can evaluate how the strength of a starting move depends on both the strategy of the computer and the opponent.

A Opening Strength under Random Play

The first player (computer AI) was forced to make its opening move on a designated cell, while all subsequent moves by both players were made randomly. The goal was to determine how the choice of the first move influences the AI agent's win rate when both sides play randomly thereafter.

This part of the experiment serves as a baseline for understanding whether certain positions offer a natural advantage, even when no strategic planning is involved. A heat map was generated to visualize the winning percentages of each of the 27 cells combined in 10 simulations.

This setup, although informative in preliminary testing, lacks depth due to the non-strategic nature of the players. Consequently, we focused our next experimental design on matchups between strategically guided agents.

B Opening Strength under Minimax and Simple Heuristic Strategy

The core of the experimental framework revolves around two types of AI strategies:

- **Minimax-based strategy** (lookAhead()): This approach evaluates all possible future moves up to a given depth using a recursive evaluation of the game tree. The AI agent simulates moves for itself and for its opponent, computing heuristic scores at the leaves and back-propagating the optimal values. It uses alpha-beta pruning to eliminate suboptimal branches early, improving performance.
- **Heuristic-based strategy** (heuristic()): This strategy is a greedy algorithm that evaluates the immediate score of each move based on certain board configurations, without simulating opponent responses or deep recursion. In the later study, we refer to it as a greedy strategy, as it does not perform a minimax search and only considers local gains.

The experiments simulate four major scenarios with varied strategy matchups to evaluate how the computer AI's strategy and human AI's strategy affect the effectiveness of different opening moves. Four different AI configurations were used in the simulations:

- **Scenario 1** - Both computer and human AI use full Minimax with alpha-beta pruning.
- **Scenario 2** - Both computer and human AI use a heuristic evaluation based solely on line availability.
- **Scenario 3** - Asymmetric strategy where the computer AI uses full minimax and human AI uses heuristic.
- **Scenario 4** - Reversed asymmetry.

For each of the 27 opening moves, 1000 matchups were run, and the resulting win rates were recorded and visualized using heat maps.

4.2 Results

The experiments reveal stark contrasts in performance between strategies and highlight the strategic value of specific positions HEATMAP.

A Random Strategy Results

The heat map for the random play scenario showed several interesting trends. Notably, slightly higher win rates were observed when the first move occurred at the four corners of the bottom and top layers. This could be attributed to the increased number of winning lines that pass through corner cells in 3D space, making them more central in terms of strategic potential.

Surprisingly, the highest win rate occurred when the first move was made at the center cell of the middle layer (1, 1, 1). This may reflect the cell's high connectivity, as it belongs to the maximum number of winning lines compared to any other cell on the board, totaling 13 unique lines. Even under random conditions, occupying a high-connectivity space increases the likelihood of forming a winning line by chance.

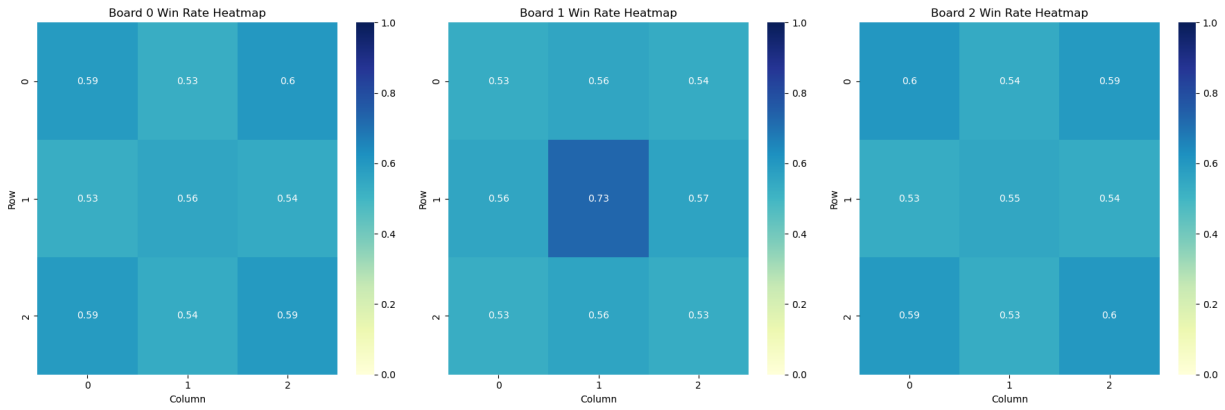

Figure 1: Random play

B Minimax and Simple Heuristic Strategy Results

The results of experiments are grouped according to the four strategic configurations:

Scenario 1: Minimax vs. Minimax
Computer AI uses *lookAhead(humanPiece, -1000, 1000)*
Human AI uses *lookAhead(computerPiece, -1000, 1000)*

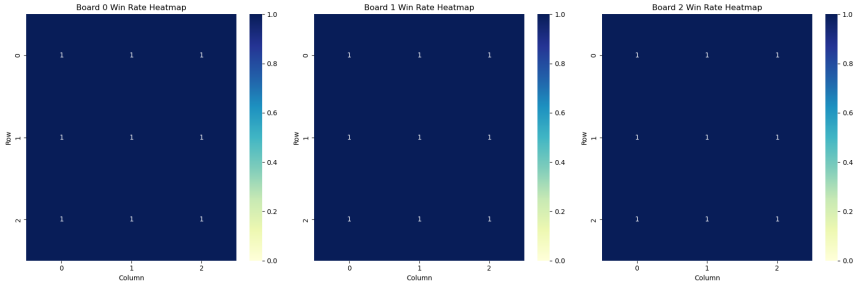

Figure 2: Minimax vs. Minimax

This setup pits two minimax agents against each other, both searching the game tree deeply and choosing moves that optimize against the other's best options. The computer AI, forced to start on each of the 27 cells in turn, achieves a win rate of 1.0 in all cases. This means that no matter where the computer AI starts, it can force a win against another minimax agent playing second. This result strongly suggests that the player who moves first has a significant advantage in 3D Tic-Tac-Toe when both players use perfect information strategies. It also affirms the correctness and strength of the minimax implementation.

Scenario 2: Greedy vs. Greedy
Computer AI uses *heuristic(computerPiece)*
Human AI uses *heuristic(humanPiece)*

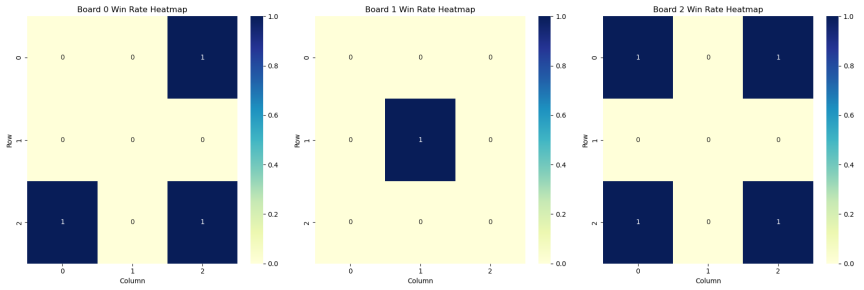

Figure 3: Greedy vs. Greedy

In this configuration, both agents use greedy strategies. Without foresight of the opponent's future actions, they base decisions on immediate heuristics. In this scenario, only 8 out of 27 positions resulted in a 100% win rate for the computer AI, specifically: (0, 0, 2) (0, 2, 0) (0, 2, 2) (1, 1, 1) (2, 0, 0) (2, 0, 2) (2, 2, 0) (2, 2, 2)

These positions correspond to the corners and the central cell of the 3D cube — positions that participate in the greatest number of potential winning lines. The rest of the starting cells showed a 0 win rate for the computer AI, highlighting how vulnerable greedy algorithms are when used without foresight. Greedy strategies tend to miss long-term threats and fall into traps set by more complex move sequences.

Scenario 3: Minimax vs. Greedy
Computer AI uses *lookAhead(humanPiece, -1000, 1000)*
Human AI uses *heuristic(humanPiece)*

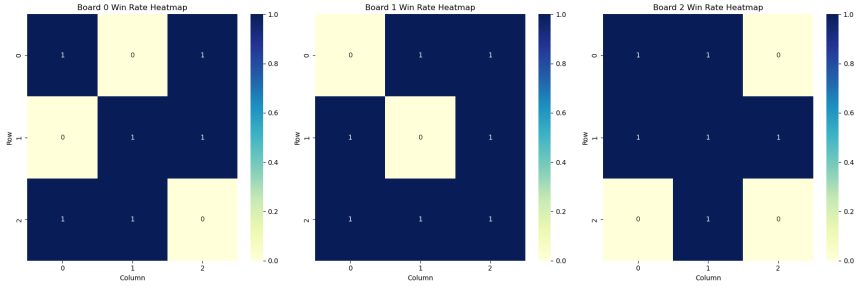

Figure 4: Minimax vs. Greedy

This asymmetrical setup tests how well the minimax-based computer AI performs against a heuristic-based (greedy) opponent. Surprisingly, there are 8 positions where the greedy human AI consistently defeats the minimax computer AI. The positions where the computer AI loses (win rate 0) are: (0, 0, 1) (0, 1, 0) (0, 2, 2) (1, 0, 0) (1, 1, 1) (2, 0, 2) (2, 2, 0) (2, 2, 2)

This outcome is particularly interesting. It suggests that although the minimax algorithm considers future possibilities, its performance can be degraded if the search depth is not sufficient to fully anticipate threats from certain aggressive greedy moves. Some cells may

lead the computer AI to paths that seem optimal in the early stages but result in suboptimal endgames due to horizon effects.

However, the remaining 19 cells yielded a win rate of 1.0 for the computer AI, showing that overall, minimax is still significantly stronger than greedy AI.

Scenario 4: Greedy vs. Minimax
Computer AI uses *heuristic(computerPiece)*
Human AI uses *lookAhead(computerPiece, -1000, 1000)*

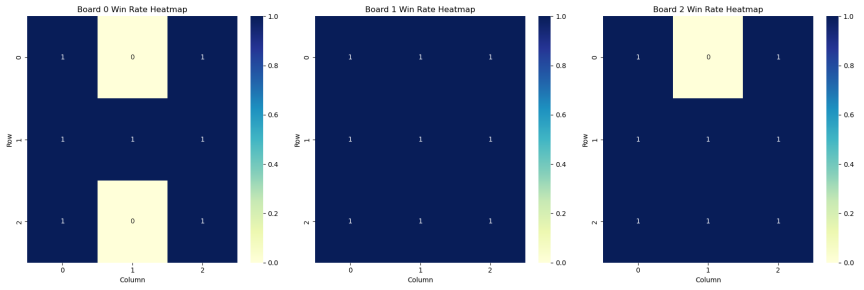

Figure 5: Greedy vs. Minimax

This reversed scenario evaluates whether a greedy Computer AI can outperform a stronger Minimax-based Human AI. Surprisingly, the Computer AI managed to win in 24 out of 27 starting positions, achieving a win rate of 1.0 in those cells. The Minimax AI was only able to consistently win in three specific cells: (0, 0, 1) (0, 2, 1) (2, 0, 1)

These cells are edge-centered positions that may favor the lookAhead algorithm by offering more immediate threats or better defensive opportunities. In all other positions, the greedy Computer AI prevailed.

This result indicates that, despite its lack of depth, the greedy heuristic was surprisingly effective at capitalizing on first-move advantages or exploiting specific board configurations. The failure of the Minimax AI in most cells may point to either limited depth in the lookahead or inherent advantages granted by certain starting positions. It also suggests that not all positions offer equal strategic value in 3D Tic-Tac-Toe—some may inherently favor simpler, aggressive strategies.

5 Analysis of the Results

The 3×3×3 Tic-Tac-Toe game offers a rich testbed for studying strategy, spatial geometry, and decision-making in a constrained environment. In this experiment, we implemented a series of gameplay scenarios using various combinations of AI strategies to explore the concept of Opening Move Strength. The goal was to determine which positions on the board naturally confer an advantage and how this interacts with different strategic approaches, particularly when the opening move is executed by either a random, greedy, or minimax-driven AI.

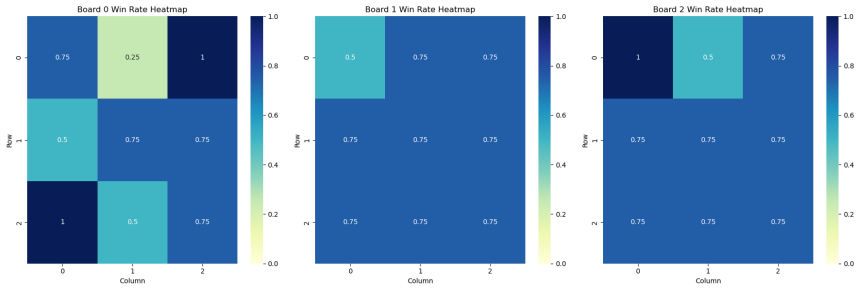

Figure 6: Combine winning rate of all four scenarios under the Minimax and simple heuristic strategy

The final heatmap image aggregates data from all gameplay scenarios, revealing which opening positions on the 3×3×3 board lead to higher win rates for the computer player. Since the computer AI always moved first, the results represent its win percentage when initiating the game from each of the 27 positions on the board. The heatmap is organized into three layers, each representing a different layer of the cube. The coloring scheme indicates the effectiveness of each opening move—darker hues correspond to lower win rates and lighter or more saturated hues indicate higher success rates.

In the Random Play scenario, both the computer and the human AIs played randomly. This scenario serves as a control group, removing all strategic influence from the game and isolating the role of board geometry. By examining which positions lead to above-average win rates in a completely stochastic environment, we can infer which openings are structurally advantageous due to their location alone.

In this setup, certain positions consistently outperformed others, despite the absence of strategy. Most notably, the center cell (1,1,1) had one of the highest win rates. This is not surprising, as the center is part of the maximum number of potential winning lines including rows, columns, pillars, and all four space diagonals. Its central location makes it an anchor for constructing multiple simultaneous threats, even without deliberate planning.

Similarly, corner positions like (0,0,0), (0,0,2), (2,2,0), and (2,2,2) also exhibited relatively high win rates. These positions participate in three orthogonal lines (x, y, and z axes) and one or more plane or space diagonals, depending on their specific placement. Their symmetrical location gives them broader influence across the 3D grid, allowing for more ways to contribute to a win.

On the other hand, edge-middle positions such as (0,1,0), (2,1,2), or (1,0,1) consistently underperformed. These positions lie between corners and the center but are only part of a few winning lines—typically two or three. As a result, they are geometrically limited in terms of their strategic potential. Even in random play, these limitations manifest as lower win rates, highlighting that some positions are inherently more powerful than others, regardless of the player's strategy.

This result affirms that the 3D geometry of the board heavily dictates positional strength. Without any tactical intent, games still trend toward favoring structurally dominant positions. In short, random play validates geometry as a meaningful factor in determining the strength of opening moves.

When we examine the outcomes across the four scenarios under the Minimax and simple heuristic strategy, we notice that Scenario 1 resulted in a flawless win rate of 1.0 across all 27 starting positions, highlighting the power of the Minimax algorithm when combined with the advantage of playing first.

Interestingly, Scenario 4 revealed a counterintuitive result. Here, the computer used a greedy strategy while the human AI used Minimax. Despite the theoretical superiority of Minimax, the greedy strategy prevailed in 24 out of 27 cells. This suggests that when the first-move advantage is combined with an aggressive positional heuristic, even a more sophisticated but reactive strategy like Minimax may be unable to counter it. It appears that some opening moves are so strong that they effectively neutralize the benefit of deeper foresight, especially in a fast-paced game like 3D Tic-Tac-Toe where the game can be won quickly.

Figure 6 illuminates the patterns in strategic strength. Board 0 features both some of the best and worst starting positions. For example, cell (0,2,0) yields a perfect win rate of 1.0, while (0,1,0) results in the lowest win rate of 0.25. This board appears to have more variable outcomes, potentially due to its edge positions being part of fewer winning lines. Board 1, the central layer, demonstrates remarkable consistency, with every cell averaging around a 0.75 win rate. This reinforces the importance of centrality in the 3D Tic-Tac-Toe strategy, as all positions in the middle layer intersect with multiple planes and directions. These trends suggest that both geometry and symmetry significantly influence win likelihoods.

6 Conclusion and Future Work

This project explored the interaction between move order, board geometry, and AI strategy in a 3×3×3 Tic-Tac-Toe game. The scenarios demonstrated that geometric advantage plays a significant role in win rates and that strategies leveraging these advantages—particularly when going first—can dramatically outperform even more sophisticated opponents. The greedy strategy, although simple, showed surprising dominance in several scenarios when paired with good positioning and turn priority. Conversely, the Minimax strategy, while theoretically superior due to its search depth and foresight, faltered when placed at a disadvantage from the start.

The implications of these findings suggest that in time-sensitive or small-space decision environments, first-move advantage and positional heuristics can outweigh deeper but slower strategic reasoning. This insight could be relevant not only for game design but also for real-world applications of AI in adversarial or competitive environments where response time is critical.

For future work, several extensions can be considered. First, introducing a more flexible evaluation function for the Minimax strategy could allow it to better identify and counter fast-win threats, particularly in greedy openings. Additionally, a more detailed study could explore second-move performance by switching the turn order and analyzing how strategies adapt when going second. Finally, introducing probabilistic strategies or reinforcement learning agents may further illuminate how AI can adapt over time to overcome geometric disadvantages and exploit opponent weaknesses.

By analyzing both the geometry and AI strategy interaction in 3D Tic-Tac-Toe, this project provides a foundation for understanding how structure and planning converge to determine success in complex, multidimensional games.

GitHub Repository Link: 3DTTT AI Analysis

References

[1] Oren Patashnik. Qubic: 4×4×4 tic-tac-toe. *Mathematics Magazine*, 53:202–216, 09 1980.

[2] Bradley D. Bogenschutz. Artificial intelligence: Implementing 3d tic-tac-toe in c++. *Summation.*, May 2010.

[3] Abubakarsidiq Makame Rajab, Nadhira Hassan, and Abdul-Rahimsidiq Rajab. Tic-tac-toe learning using artificial neural networks. *International Journal of Engineering and Information Systems (IJEAIS)*, Vol. 3:: 1–6, 01 2020.

[4] Husni Thamrin and Ramadhana Pamungkas. Algoritma minimax untuk game tic tac toe yang menantang. *Indonesian Journal of Computer Science*, 12, 06 2023.

[5] Michiel Steeg, Madalina Drugan, and Marco Wiering. Temporal difference learning for the game tic-tac-toe 3d: Applying structure to neural networks. 12 2015.

[6] Mohaned Abu Dalffa, Bassem S. Abu-Nasser, and Samy S. Abu-Naser. Tic-tac-toe learning using artificial neural networks. *International Journal of Engineering and Information Systems (IJEAIS)*, 3(2):9–19, 2019.

[7] Sanjiv Sridhar and Shaumik Shetty. Classification of tic-tac-toe game using deep neural network. 05 2021.

[8] Nested Software. Tic tac toe with the minimax algorithm, June 2019. Accessed: 2025-04-25.